

# Logging

bb6301c6-4e62-4bbf-b869-33b533287481

## 1. Introducere

Pentru logarea mesajelor de debug în drivere i aplicaai este recomandat folosirea *Windows software trace preprocessor (WPP)* ca alternativ la scrierea mesajelor într-un fiier i/sau afiarea lor în consol. WPP este bazat pe infrastructura [Event Tracing for Windows](#) i este suportat începând cu Windows 2000.

Pentre avantajele semnificative ale WPP se numr urmtoarele:

- String-urile sunt extrase de un preprocesor înainte de compilarea codului i nu ajung în binare. În locul lor se vor genera mesaje binare care pot fi colectate oricând (\*.ETL), dar pentru citirea lor este necesar PDB-ul sau TMF.
- Impactul de performanta adus de logging este nesemnificativ când logging-ul este oprit. Se va face doar o comparaie i se evit situaiile în care logarea sincronizeaz codul.
- Doar crearea mesajului binar este sincron. Transmiterea i consumarea mesajului (i.e. scrierea în fiier sau transmiterea ctre KD) sunt asincrone.
- Logurile pot fi incluse i în builduri de release, i activate doar dac se dorete.
- Logurile se pot activa individual per component i se poate specifica un nivel minim de severitate dorit
- Se colecteaz automat detalii despre loguri (fiier, linie, component, etc)

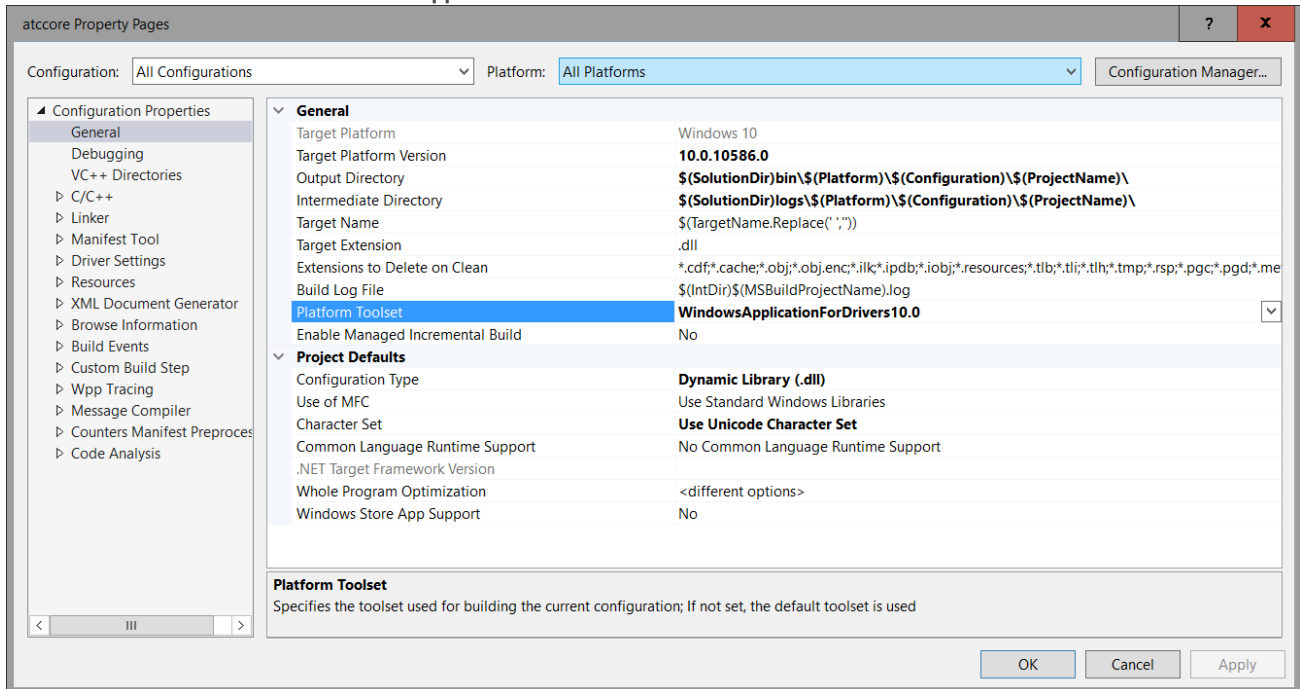
Funcionalitatea WPP este împrit în 3 componente: providers, controllers i consumers. Providerii sunt componentele software care genereaz loguri, controllerele reprezint infrastructura wpp i uneltele de configurare, iar consumerii sunt sesiunile de logare i uneltele care preiau mesajele. Pentru a colecta logurile trebuie creat un consumator i activai provideri pentru acel consumator.

Urmatoarele seciuni prezint pe scurt cum se folosete WPP.

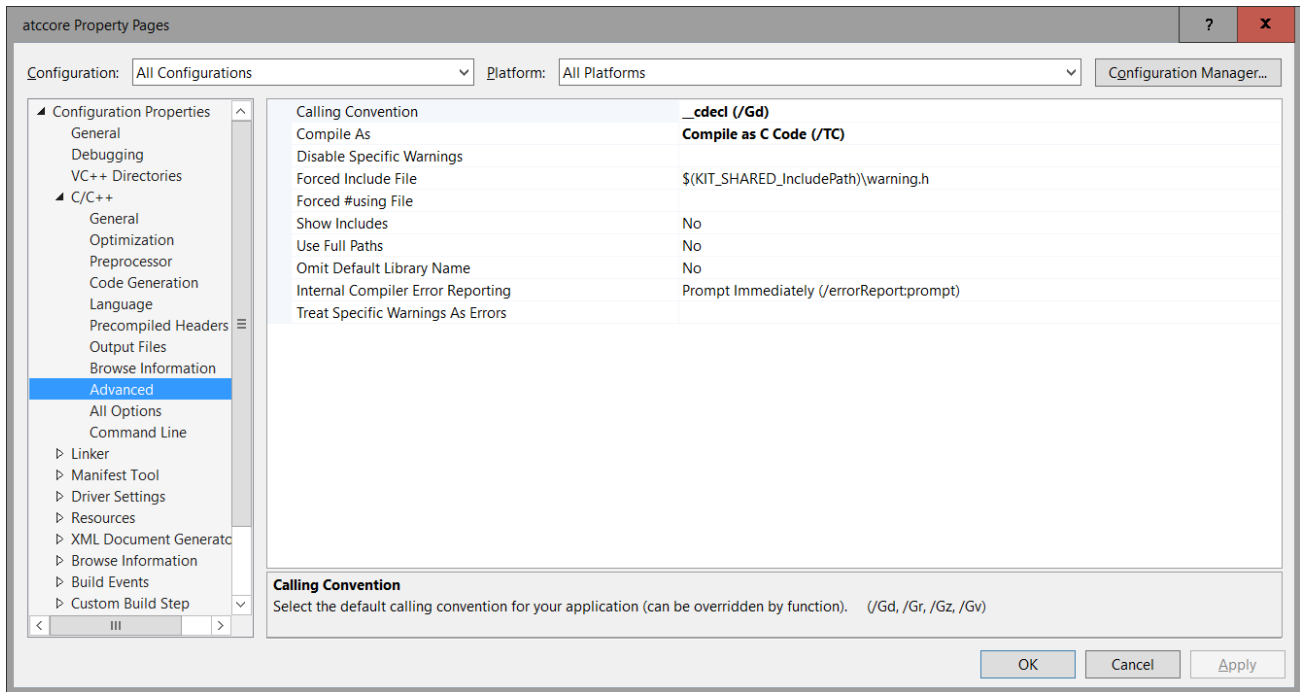
## 2. Configurri WPP specifice pentru aplicaai User Mode

Folosirea WPP nu difer în cadrul codurilor KM/UM. Dar pentru a folosi WPP în cadrul aplicaaiilor User Mode sunt necesari câiva pai suplimentari. Pe lâng SDK, trebuie instalat i WDK. În plus trebuie efectuate urmtoarele schimbri la nivel de proiect:

- **Platform Toolset** trebuie setat la **Windows Application For Drivers**.



- Schimbarea toolsetului duce la schimbarea conveniei de apel a funciilor de la `__cdecl` la `__stdcall`. Dacă se dorește, se poate trece manual la convenia inițială.



### 3. Configurarea WPP

Următoarele sunt valide indiferent dacă ne aflăm în KM sau UM.

#### 3.1. Generarea unui fișier de configurare pentru WPP

Configurările WPP specifice proiectului vor fi stocate într-un header similar celui de mai jos:

```

                                trace.h
#ifndef _TRACE_H_
#define _TRACE_H_
//
// Copyright (C) 2015 BitDefender S.R.L.
// Author(s) : Radu PORTASE(rportase@bitdefender.com)
//
//
// WPP Control GUID for MyDriver: bb6301c6-4e62-4bbf-b869-33b533287481
//
#define WPP_CONTROL_GUIDS \
    WPP_DEFINE_CONTROL_GUID( \
        MyDriverTraceGuid, (bb6301c6-4e62-4bbf-b869-33b533287481), \
        WPP_DEFINE_BIT(TRACE_FLAG_DRIVER) /* bit 0 = \
0x00000001 */ \
        WPP_DEFINE_BIT(TRACE_FLAG_FILE_FILTER) /* bit 1 = \
0x00000002 */ \
    )
//
// Because we are using our custom logging function DrvLogMessage(Flags, \
Level, Message, ...) we need to define the following macros
//

```

```

#define WPP_LEVEL_FLAGS_LOGGER(lvl, flags) WPP_LEVEL_LOGGER(flags)
#define WPP_LEVEL_FLAGS_ENABLED(lvl, flags) (WPP_LEVEL_ENABLED(flags) &&
WPP_CONTROL(WPP_BIT_ ## flags).Level >= lvl)
//
// Here we define our logging functions
//
// begin_wpp config
//
// Functions for logging driver related events
//
// FUNC DrvLogTrace{LEVEL=TRACE_LEVEL_VERBOSE, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
// FUNC DrvLogInfo{LEVEL=TRACE_LEVEL_INFORMATION, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
// FUNC DrvLogWarning{LEVEL=TRACE_LEVEL_WARNING, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
// FUNC DrvLogError{LEVEL=TRACE_LEVEL_ERROR, FLAGS=TRACE_FLAG_DRIVER}(MSG,
...);
// FUNC DrvLogCritical{LEVEL=TRACE_LEVEL_CRITICAL, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
//
// Functions for logging file filter related events
//
// FUNC FltLogTrace{LEVEL=TRACE_LEVEL_VERBOSE,
FLAGS=TRACE_FLAG_FILE_FILTER}(MSG, ...);
// FUNC FltLogInfo{LEVEL=TRACE_LEVEL_INFORMATION,
FLAGS=TRACE_FLAG_FILE_FILTER}(MSG, ...);
// FUNC FltLogWarning{LEVEL=TRACE_LEVEL_WARNING,
FLAGS=TRACE_FLAG_FILE_FILTER}(MSG, ...);
// FUNC FltLogError{LEVEL=TRACE_LEVEL_ERROR, FLAGS=TRACE_FLAG_FILE_FILTER}
(MSG, ...);
// FUNC FltLogCritical{LEVEL=TRACE_LEVEL_CRITICAL,
FLAGS=TRACE_FLAG_REGISTRY_FILTER}(MSG, ...);
//
//
// end_wpp
//

#endif//_TRACE_H_

```

Urmtoarele paragrafe descriu componentele acestui fiier.

Pentru orice proiect care foloseste WPP trebuie definit un GUID care reprezinta ID-ul unic al unui provider pentru logging i componentele pentru care se face logging-ul. Aceasta se face cu o constructie de forma:

```

#define WPP_CONTROL_GUIDS
    WPP_DEFINE_CONTROL_GUID(
        MyDriverTraceGuid, (bb6301c6-4e62-4bbf-b869-33b533287481),
        WPP_DEFINE_BIT(TRACE_FLAG_DRIVER)                /* bit 0 =
0x00000001 */ \
        WPP_DEFINE_BIT(TRACE_FLAG_FILE_FILTER)           /* bit 1 =
0x00000002 */ \
    )

```

Astfel am definit un provider care are doua subcomponente pentru care logging-ul poate fi activat separat.

Dac exista mai multi provideri (de exemplu folosim i o librie static) atunci constructia de mai sus poate sa contina o lista de GUID-uri. De exemplu:

```
#define WPP_CONTROL_GUIDS \
    WPP_DEFINE_CONTROL_GUID(SharedStaticLibs, (81b20feb, 73a8, 4b62, 95bc, 354477c97a6f), \
        WPP_DEFINE_BIT(ComponentA) \
        WPP_DEFINE_BIT(Component2) \
        WPP_DEFINE_BIT(ComponentIII) \
    ) \
    WPP_DEFINE_CONTROL_GUID(AppSpecificFlags, (81b20fec, 73a8, 4b62, 95bc, 354477c97a6f), \
        WPP_DEFINE_BIT(EntryExit) \
        WPP_DEFINE_BIT(Initialization) \
        WPP_DEFINE_BIT(MemoryAllocations) \
    )
```

Deja doar prin definirea GUID-ului se poate folosi [DoTraceMessage](#) pentru a loga diverse mesaje pentru componentele specificate. Dezavantajul este c nu putem specifica nivelul de severitate al mesajelor (error, warning, info, etc). Pentru aceasta trebuie sa definim macroui proprii de logare. WPP cauta comentarii care incep cu *// begin\_wpp* config pentru a gsi datele de configurare. Funciile de logare se definesc folosind FUNC:

```
// begin_wpp config
//
// FUNC DrvLogTrace{LEVEL=TRACE_LEVEL_VERBOSE, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
// FUNC DrvLogInfo{LEVEL=TRACE_LEVEL_INFORMATION, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
// FUNC DrvLogWarning{LEVEL=TRACE_LEVEL_WARNING, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
// FUNC DrvLogError{LEVEL=TRACE_LEVEL_ERROR, FLAGS=TRACE_FLAG_DRIVER}(MSG,
...);
// FUNC DrvLogCritical{LEVEL=TRACE_LEVEL_CRITICAL, FLAGS=TRACE_FLAG_DRIVER}
(MSG, ...);
//
// end_wpp
//
```

Deja am declarat macroui noi pentru logging, cu nivele de severitate corespunzatoare. Mai trebuie s descriem cum se folosesc parametrii LEVEL i FLAGS ai macrouilor pentru a manipula logging-ul. Facem aceasta folosind WPP\_\*\_LOGGER i WPP\_\*\_ENABLED, unde \* se inlocuieste în ordine cu numele parametrilor suplimentari (**case sensitive**) ai macrouilor definite:

```
#define WPP_LEVEL_FLAGS_LOGGER(lvl, flags) WPP_LEVEL_LOGGER(flags)
#define WPP_LEVEL_FLAGS_ENABLED(lvl, flags) (WPP_LEVEL_ENABLED(flags) &&
WPP_CONTROL(WPP_BIT_ ## flags).Level >= lvl)
```

Comportamentul default pentru WPP este de a linka executabilul cu advapi32.dll. Daca acest lucru nu este dezirabil, exista posibilitatea de a utiliza doar functii din ntdll prin definirea macro-ului: WPP\_USE\_NTDLL\_FUNCTIONS in fisierul trace.h:

```
#define WPP_USE_NTDLL_FUNCTIONS
```

### 3.2. Iniializarea i oprirea framework-ului WPP

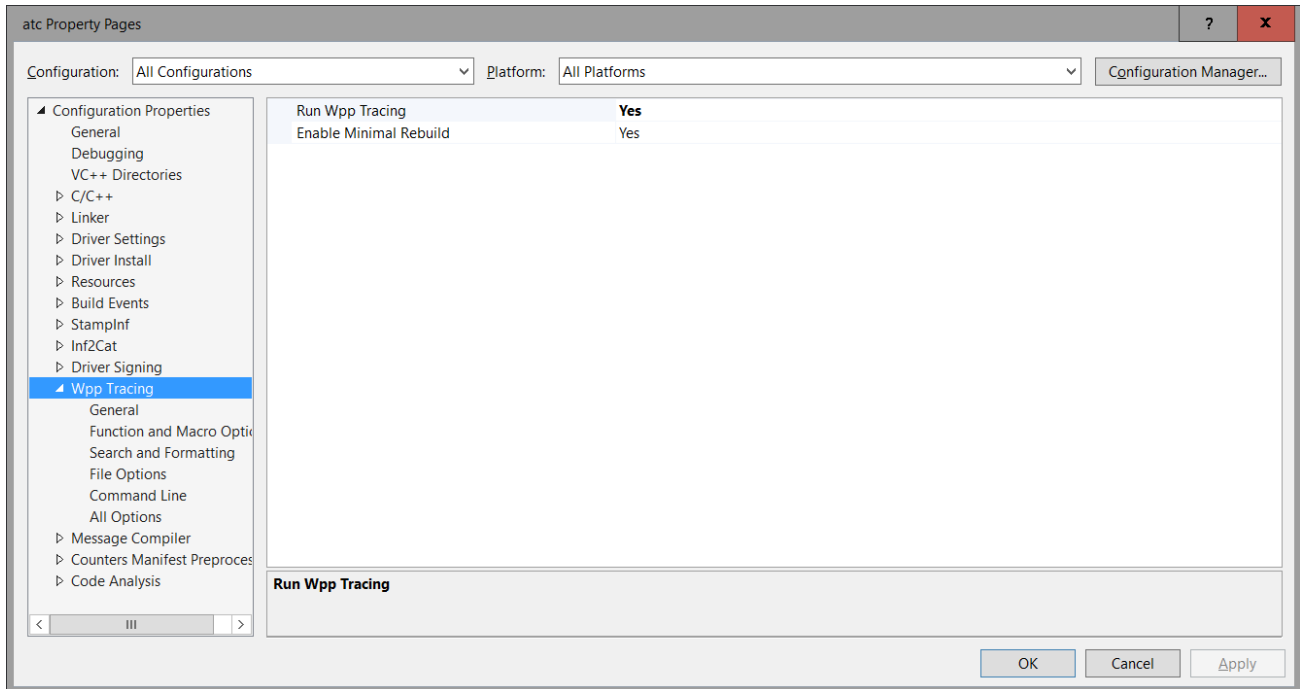
Pentru a folosi WPP, trebuie inițializat la entry point-ul codului cu ajutorul macroului `WPP_INIT_TRACING`. De exemplu DriverEntry pentru drivere, etc.

Înainte de descrcare trebuie apelat macroul `WPP_CLEANUP`.

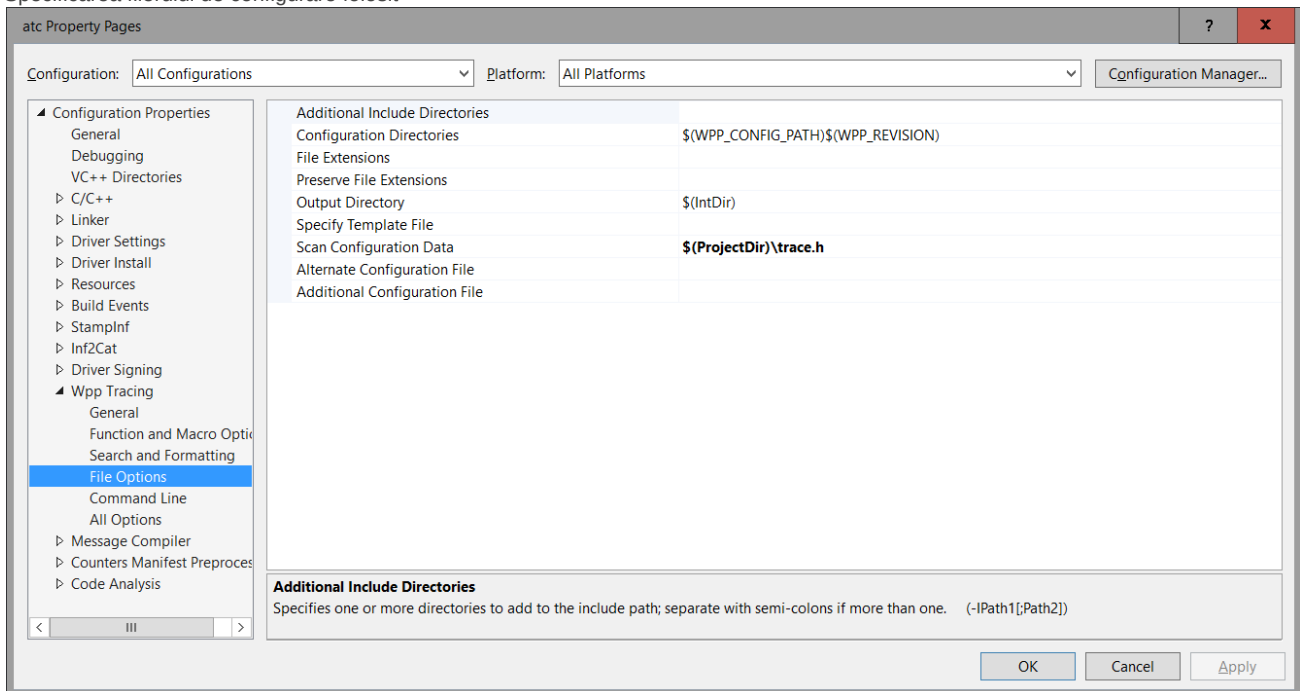
### 3.3. Configurarea proiectelor Visual Studio pentru a folosi WPP

Pentru Visual Studio-uri mai noi decât Visual Studio 2012 activarea WPP se face în doi pași simpli:

- Activarea WPP



- Specificarea fiierului de configurare folosit



### 3.4. Folosirea WPP în fierele surs

În cadrul surselor din care dorim să generăm loguri trebuie să includem fiierul de configurare (*trace.h*). În urma rulării WPP sunt generate pentru fiecare surs din proiect fiere *<nume\_surs>.tmh* care conțin detaliile despre logurile care se fac din sursa corespunzătoare. Pentru a putea folosi funcțiile de logare WPP trebuie ca în fiecare fiier surs să includem după header-ul de configurare și fiierul .tmh generat de WPP. De exemplu fiierul source.c ar trebui să înceapă cu:

```

source.c

#include "trace.h"
#include "source.tmh"

```

Acum se pot genera loguri folosind funcțiile definite anterior în cadrul headerului de configurare. Acestea funcționează cu sintaxa standard printf.

## 4. Colectarea logurilor

Pentru accesarea logurilor WPP avem la dispoziție mai multe aplicații.

### 4.1. Logman

**Logman** vine împreună cu Windows și permite logarea în fiier.

#### a. Se creează o sesiune de logging

```

logman create trace "MyDriver.Sys" -ets -p {xx-x-x-x-xxx} 0xffffffff 0xff -
o "C:\\mylog.etl"

```

GUID-ul trebuie să fie cel declarat în WPP\_DEFINE\_CONTROL\_GUID. 0xffffffff reprezintă flag-urile de tracing activate și 0xff este nivelul de logare

#### b. Se porneste sesiunea

```

logman start "MyDriver.Sys"

```

## 4.2. Tracelog

WDK. **tracelog** permite logarea în fiier sau ctre Kernel Debugger.

```
tracelog -start trace_name [-f file_path[-append]] [-kd]
tracelog -stop trace_name
tracelog -enable trace_name -guid #xx-x-x-xxx -flags 0xflags -level lvl
tracelog -disable trace_name -guid #xx-x-x-xxx
```

Exemple pentru pornirea/oprirea sesiunii i înregistrarea providerilor.

```
tracelog.exe -addautologger MyDriver-guid #bb6301c6-4e62-4bbf-b869-
33b533287481 -sessionguid #bb6301c6-4e62-4bbf-b869-33b533287481 -level 4 -
flag 0xffff -rt -kd -f %cd%\MyDriver.etl -append
tracelog.exe -addautologger trace_session [-sessionguid #xx-x-x-xxx] -guid
#xx-x-x-xxx [-level lvl] [-flag 0xflags] [-rt] [-kd] [-f file_path [-
append]]
```

Înregistrează un logger persistent. Setările sunt stocate în registri (HKEY\_LOCAL\_MACHINE\SYSTEM\CurrentControlSet\Control\WMI\AutoLogger\), astfel logarea rămâne activ după reboot.

- -f -> local etl file (full path) where logs are stored.
- -kd -> direcionează logurile către Kernel Debugger
- -rt -> log realtime
- -guid -> GUID din trace.h
- -sessionguid -> guid pentru sesiunea de log. Poate lipsi dar MS recomand o valoare.
- -flags -> componentele logate
- -level -> nivelul minim de severitate

```
tracelog -flush trace_name
```

-> Flush logging buffers. (Implicit se realizează automat o dată pe secundă.)

## 4.3. TraceView

WDK. Unealtă GUI care permite vizualizarea logurilor în timp real i salvarea lor în fiier. Logurile sunt afișate în format tabelar i permite sortarea după componente, severitate, fiier, linie, etc.